Day 2_Interview Questions

Q1. What happens in an LLM call?
You send a request to the server (Groq) with your API key, exact model name, and a list of messages (role + content). The server runs the model, predicts tokens, and returns a response object. You read the answer from choices[0].message.content.

Q2. What is a token?
A chunk of text — roughly 4 characters or about three-fourths of a word. It's a subword unit, not a full word ("unbelievable" can be 3 tokens). Trap: "1 token = 1 word" is wrong.

Q3. Why is the answer under choices, and why an array?
A model can be asked for multiple answers (the n parameter). Default n=1, so you read choices[0]. It's an array to support the multiple-answers case.

Q4. What's in usage and why care?
prompt_tokens, completion_tokens, total_tokens. You pay per token (input + output) and each model has a context window limit. Long chats quietly raise cost and can overflow the window.

Q5. Why does each message need a role?
Roles give context. user = you, assistant = past replies, system = behavior rules. Without roles the model can't tell who said what ("what is his age?" becomes meaningless). A conversation is just an ordered list of role-tagged messages.

Q6. Is the LLM stateless? Does it remember my last message?
Yes, stateless — it remembers nothing between calls. Memory is an illusion: you resend the whole history each call, which is why long chats cost more. Follow-up: "How does ChatGPT remember?" → the app stores your history and replays it.

Traps

- content vs prompt: the dict key must be content; prompt errors (API rule).
- Model name is exact — one wrong letter fails. No fuzzy matching.
- Messages must be a list, even for one message.
- "Why Groq not OpenAI?" → free and very fast inference. Concept is identical; only client and model name change.